

HW1

B12901035 Zheng Yu-Yan

1 TF compared with FT

- **Advantage:** Results from time-frequency analysis can show the variation of the spectrum over time, while the results of the Fourier transform will not contain time information, but only frequency components. In other words, we can observe the instantaneous frequency from the TF analysis. Moreover, time-frequency analysis makes digital signal processing more adaptive. For example, the bandwidth, as well as sampling points, can vary with time thanks to TF analysis, which allows for more efficient and effective further applications.
- **Disadvantage:** Compared with the Fourier transform, the time-frequency analysis is more time-consuming as it is computationally demanding. According to the course notes, under the same number of sampling points (the same resolution), the total time required for TF analysis is approximately 100 times that of the Fourier transform. Moreover, time-frequency analysis sacrifices frequency resolution due to the uncertainty principle, and the result always involves a trade-off between time resolution and frequency resolution.

2 Time-invariant instantaneous frequency

$$x(t) = \exp[j(a_0 + a_1t + a_2t^2 + a_3t^3 + \dots)] = \exp[j \cdot \phi(t)]$$

Instantaneous frequency of $x(t)$ is:

$$f(t) := \frac{1}{2\pi} \frac{d}{dt} \phi(t) = \frac{1}{2\pi} (a_1 + 2a_2t + 3a_3t^2 + \dots)$$

Only when **1.** $\phi(t) = a_1t + a_0, a_{k \geq 2} = 0$ or **2.** $\phi(t) = a_0, a_{k \geq 1} = 0$ and a_1, a_0 are also **time invariant**, instantaneous frequency doesn't vary with time. i.e. $f(t) \in Constant$

3 Chirp signal may be generated under...

- (i) **Wireless Communication:** In chirp spread spectrum (CSS), information is modulated onto a chirp that sweeps its frequency.

$$s(t) = \cos\left(2\pi\left(f_c t + \frac{B}{2T}t^2\right)\right), \quad 0 \leq t \leq T$$

where f_c is the carrier, B is the bandwidth, and T is the symbol duration.
e.g. LoRa uses up-chirp and down-chirp signals to represent digital bits.

- (ii) **Radar System:** In the FMCW radar, the transmitted chirp is reflected from a target. The received signal is delayed, producing a beat frequency proportional to distance:

$$s_{\text{tx}}(t) = \cos\left(2\pi\left(f_0 t + \frac{K}{2}t^2\right)\right), \quad s_{\text{rx}}(t) = \cos\left(2\pi\left(f_0(t - \tau) + \frac{K}{2}(t - \tau)^2\right)\right)$$

where K is chirp rate, τ is delay, and the beat frequency $f_b = K\tau$ reveals target range.

- (iii) **Acoustics:** the 2nd, 3rd, 4th, and 5th tones of Chinese words.

4 Who varies the number of sampling points

Let the original signal be $x(t) \xrightarrow{FT} X(j\omega)$, the number of sampling points = area of frequency-time plot.

For real signal, according to sampling theory: $\Delta_t \leq \frac{1}{2B}$, B : max positive frequency

(i) Scaling: **No**. With scaling factor σ :

$$X_\sigma(j\omega) := \mathcal{F}[x(\sigma t)] = \int_{-\infty}^{\infty} x(\sigma t) e^{-j\omega t} dt = \frac{1}{|\sigma|} \int_{-\infty}^{\infty} x(\sigma t) e^{-j\frac{\omega}{\sigma} \sigma t} d(\sigma t) = \frac{1}{|\sigma|} X(j\frac{\omega}{\sigma})$$

Therefore, the size of time range is scaled by $\frac{1}{\sigma}$, while the size of frequency range is scaled by σ , the area of frequency-time plot will not change.

(ii) Time-shifting: **No**. With time offset τ :

$$X_\tau(j\omega) := \mathcal{F}[x(t-\tau)] = \int_{-\infty}^{\infty} x(t-\tau) e^{-j\omega t} dt = e^{-j\omega\tau} \int_{-\infty}^{\infty} x(t-\tau) e^{-j\omega(t-\tau)} d(t-\tau) = e^{-j\omega\tau} X(j\omega)$$

Therefore, both the size of time range and the size of frequency range will not change, the area of frequency-time plot remains the same.

(iii) Multiplied by t : **Yes**:

$$X_m(j\omega) := \mathcal{F}[tx(t)] = \int_{-\infty}^{\infty} tx(t) e^{-j\omega t} dt = j \frac{d}{d\omega} [X(j\omega)]$$

Therefore, the frequency pattern of this new signal can be derived from differentiation w.r.t. frequency, which theoretically won't affect the frequency range, however the area under spectrogram will probably change, the number of sampling points in turn changes.

(iv) multiplying by $\exp(jt)$: **No**:

$$X_e(j\omega) := \mathcal{F}[\exp(jt)x(t)] = \int_{-\infty}^{\infty} x(t) e^{-j(\omega-1)t} dt = X(j(\omega-1))$$

Therefore, the frequency range is shifted by $1/2\pi$, which implies that bandwidth will not change, and the whole area of frequency-time plot stays the same. The number of sampling points required stays the same.

5 How to compute frequency by zero crossings

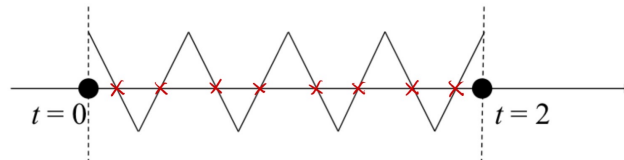


Figure 1: Example of zero crossing, the red “x” symbol represents a zero crossing

$$\text{frequency} = \frac{\text{number of zero crossings}}{2 \times (\text{time duration})}$$

From the above example, we can calculate the signal's frequency as $\frac{8}{2 \times 2} = 2(\text{Hz})$

6 Useful discrete wavelet transform

1D-DWT:

$$x_{1,L}[n] = \sum_k x[2n - k]g[k], \quad g[k] : \text{lowpass filter}$$

$$x_{1,H}[n] = \sum_k x[2n - k]h[k], \quad h[k] : \text{highpass filter}$$

2D-DWT:

$$x[m, n] \rightarrow x_{1,L}[m, n], x_{1,H_1}[m, n], x_{1,H_2}[m, n], x_{1,H_3}[m, n]$$

a) Image compression: 2D discrete wavelet transform mainly generate 4 components from the original signal(image) $x[m, n]$:

- $x_{1,L}$: low frequency component
- x_{1,H_1} : horizontal high frequency
- x_{1,H_2} : vertical high frequency
- x_{1,H_3} : high frequency on in both directions

It's worth noticing that these four components has the same size of $\frac{(m+L-1)(n+L-1)}{4}$, and the total number of bits used before and after is approximately the same ($\propto mn$) since $L \ll m, n$. Therefore, unlike STFT, wavelet transform won't require large memory to store generated results.

Generally, for an image its low frequency part accounts for the largest proportion compared with the remaining high frequency parts because high frequency components represent details, and x_{1,H_3} may even contain unnecessary noise.

In conclusion, $x_{1,L}$ is sufficient enough to carry semantic information, or represent the "big picture", of the original picture $x[m, n]$, while $x_{1,L}$ only requires $\approx 1/4$ number of bits to store, which indicates that **2D-DWT has the potential on image compression**. We can further continue the compression process by doing wavelet transform on $x_{1,L}$ to achieve higher compression rate.

For example, JPEG2000 relies on wavelet transform to achieve higher compression rate on both gray scale and colorful images than JPEG using DCT.

b) Directional edge detection: there are 4 components generated from 2D-DWT

$$x \xrightarrow{g[n]} \xrightarrow{\downarrow 2} \xrightarrow{g[m]} \xrightarrow{\downarrow 2} x_{1,L}$$

$$x \xrightarrow{g[n]} \xrightarrow{\downarrow 2} \xrightarrow{h[m]} \xrightarrow{\downarrow 2} x_{1,H_1}$$

$$x \xrightarrow{h[n]} \xrightarrow{\downarrow 2} \xrightarrow{g[m]} \xrightarrow{\downarrow 2} x_{1,H_2}$$

$$x \xrightarrow{h[n]} \xrightarrow{\downarrow 2} \xrightarrow{h[m]} \xrightarrow{\downarrow 2} x_{1,H_3}$$

As we can see, x convolves with high-pass filter h along m-axis, which means component x_{1,H_1} will contain the information recording **horizontal rapid change = horizontal edge**. On the other hand, component x_{1,H_2} is generated through convolution with h along n-axis, which indicates that it records **vertical rapid change = vertical edge**. For the last high-frequency component x_{1,H_3} , it carries rapid change in both directions, while we often regard it as **corner** or **noise**.

In conclusion, **2D-DWT** has great potential on **directional edge detection**.

7 rec-STFT

7.(a)

rec-STFT:

$$X(t, f) = \int_{t-B}^{t+B} x(\tau) e^{-j2\pi f\tau} d\tau$$

The effect towards resolution regarding window width B can be shown by 2 examples:

- $x(t) = \delta(t)$: $X(t, f) = \int_{t-B}^{t+B} \delta(\tau) e^{-j2\pi f\tau} d\tau = 1$ when $-B < t < B$, $X(t, f) = 0$ otherwise
Ideal condition of the time frequency spectrum for impulse $\delta(t)$ is considered as non-zero frequency components only occurs at $t = 0$. Therefore, with $B \rightarrow 0$, the result of rec-STFT will approach ideal situation along time axis.
- $x(t) = 1$: $X(t, f) = \int_{t-B}^{t+B} e^{-j2\pi f\tau} d\tau = 2B \text{sinc}(2Bf) e^{-j2\pi ft}$
Ideal condition of the time frequency spectrum for constant signal is considered as zero frequency at all time. Therefore, when consider $|X(t, f)| = 2B \text{sinc}(2Bf) = 0$ where $f = n/2B$, with B becomes larger, the result of rec-STFT will approach ideal situation along frequency axis.

In conclusion, with smaller window width B resolution in time domain is higher while resolution in frequency domain is lower, vice versa.

7.(b)

Asymmetric window indicates that different lengths of information of past and future are required for time frequency analysis. Therefore, asymmetric windows have advantage on **real-time processing**, since while we have to keep the window size large enough for decent frequency resolution, we can narrow one side of the window so that we won't have to wait long time for calculation of current data point. The dependency on future data can in turn be alleviated.

7.(c)

For the first reason, rectangular window will introduce excessive high frequency components on boundaries, which causes distortion and inaccuracy in time-frequency analysis. Gaussian window, on the other hand, has smoother decay on boundaries.

Secondly, among all functions, the Gaussian function has the advantage that the area in time-frequency distribution is minimal. While the window size of rectangular will seriously affect the resolution on time and frequency domains, the Gaussian function is the eigenfunction of Fourier Transform and thus Gabor transform will have symmetric properties on both domains. In addition, we can prove that Gaussian function satisfies the lower bound of Henseinberg's uncertainty principle, which makes Gabor transform more ideal than rec-STFT for it can take better care of both domains' clarity.

From the other perspective, according to Central Limit Theorem, if a sequence of iid random variables X_i with zero mean and limited variance, $Y_n = \sum_{i=0}^n X_i$, $\lim_{n \rightarrow \infty} Y_n \sim \text{Gaussian}$. If we let the pdf of $X_i: f_{X_i}$ as uniform distribution between $\pm B$ representing the rectangular window, then we can derive that $f_{Y_n} = f_{X_1+X_2+\dots+X_n} = f_{X_1} * f_{X_2} * \dots * f_{X_N}$, $\lim_{n \rightarrow \infty} f_{Y_n} = \text{Gaussian}$. Therefore, the Gaussian window will be the convolution result of infinite rectangular windows. When turn to frequency domain, the Fourier transform of a single rectangular window is sinc function. Then the Fourier transform of Gaussian window can be considered as infinite sinc functions being multiplied together, so the main lobe broadens, while high-frequency ripples are suppressed, which is consistent

with the fact that the Fourier transform of Gaussian is still Gaussian. From the second example in (a) part, with larger main lobe and no ripples of higher frequency, the time frequency result will be more "ideal".

8 Gabor transform

$$\mathcal{G}(t, f) = \int_{-\infty}^{\infty} e^{-\pi\sigma(\tau-t)^2} e^{-j2\pi f\tau} x(\tau) d\tau \quad (1)$$

$$= \int_{-\infty}^{\infty} e^{-\pi\sigma(\tau-t)^2} e^{-j2\pi f\tau} e^{-\pi\tau^2} d\tau ; \sigma = 1 \quad (2)$$

$$= \int_{-\infty}^{\infty} e^{-\pi[\tau^2 - 2\tau t + t^2 + \tau^2]} e^{-j2\pi f\tau} d\tau \quad (3)$$

$$= e^{-\pi t^2} \int_{-\infty}^{\infty} e^{-2\pi\tau^2 + 2\pi\tau(t-jf)} d\tau ; -2\pi\tau^2 + 2\pi\tau(t-jf) = -2\pi\left(\tau - \frac{t-jf}{2}\right)^2 + \frac{\pi}{2}(t-jf)^2 \quad (4)$$

$$= e^{-\pi t^2} e^{\frac{\pi}{2}(t-jf)^2} \int_{-\infty}^{\infty} e^{-2\pi\left(\tau - \frac{t-jf}{2}\right)^2} d\tau ; \int_{-\infty}^{\infty} e^{-a(t-\mu)^2} = \sqrt{\frac{\pi}{a}} \quad (5)$$

$$= \frac{1}{\sqrt{2}} e^{-\pi t^2} e^{\frac{\pi}{2}(t-jf)^2} \quad (6)$$

$$= \frac{1}{\sqrt{2}} \exp\left(-\frac{\pi}{2}(t^2 + f^2)\right) e^{-j\pi t f} \quad (7)$$

9 Python simulation

For the signal with instantaneous frequency: $\pm(at^2 + bt + c)$

$$phase = \int (at^2 + bt + c) dt = \frac{at^3}{3} + \frac{bt^2}{2} + ct$$

$$signal\ x = \cos\left(2\pi\left(\frac{at^3}{3} + \frac{bt^2}{2} + ct\right)\right)$$

Example:

$$f(t) = \pm(-45t^2 + 520t + 20), \quad T = 10, Fs = 44100$$

Screenshots/Images are on the next page.

10 Bonus

$$x_2(t) = \cos(12\pi t - 0.1\pi t^2) = \text{Re}\left\{\frac{1}{2}[e^{j(-12\pi t + 0.1\pi t^2)} + e^{j(12\pi t - 0.1\pi t^2)}]\right\}$$

Phase: $\phi(t) = \pm(12\pi t - 0.1\pi t^2)$

Instantaneous frequency: $f(t) := \frac{1}{2\pi} \frac{d}{dt} \phi(t) = \pm(6 - 0.1t)(\text{Hz})$

```

gwave.py x output.wav frequency_vs_time.png
gwave.py > gwave
1 import numpy as np
2 from scipy.io.wavfile import write
3 from scipy.signal import stft
4 import matplotlib.pyplot as plt
5
6 def gwave(a, b, c, T, Fs, sign=1, filename="output.wav"):
7     # time axis
8     t = np.linspace(0, T, int(Fs * T), endpoint=False)
9
10    # f
11    f = sign * (a * t**2 + b * t + c)
12
13    # integral of f
14    phase = 2 * np.pi * (a * (t**3) / 3 + b * (t**2) / 2 + c * t)
15
16    # cosine
17    x = np.cos(phase)
18
19    x_int16 = np.int16(x / np.max(np.abs(x)) * 32767)
20    write(filename, Fs, x_int16)
21    print(f"WAV file saved: {filename}")
22
23    # -----
24    # f(t)
25    plt.figure(figsize=(8,4))
26    plt.plot(t, f, color='blue')
27    plt.xlabel("Time [s]")
28    plt.ylabel("Frequency [Hz]")
29    plt.title("Instantaneous Frequency")
30    plt.grid(True)
31    plt.tight_layout()
32    plt.savefig("frequency_vs_time.png")
33    print("Frequency vs time plot saved: frequency_vs_time.png")
34    plt.close()
35
36    # -----
37    # STFT
38    f_stft, t_stft, Zxx = stft(x, Fs, nperseg=1024)
39    magnitude = np.abs(Zxx)
40
41    f_max = np.max(np.abs(f)) * 1.1 # 10% margin
42
43    plt.figure(figsize=(8,4))
44    plt.pcolormesh(t_stft, f_stft, magnitude, shading='gouraud')
45    plt.ylim(0, f_max)
46    plt.ylabel("Frequency [Hz]")
47    plt.xlabel("Time [s]")
48    plt.title("Spectrogram (STFT)")
49    plt.colorbar(label='Amplitude')
50    plt.tight_layout()
51    plt.savefig("spectrogram.png")
52    print("Spectrogram plot saved: spectrogram.png")
53    plt.close()
54
55    if __name__ == "__main__":
56        print("Frequency: 1(at^2+bt+c) Hz, t in seconds")
57        a = float(input("Enter a (default -45): ") or -45)
58        b = float(input("Enter b (default 520): ") or 520)
59        c = float(input("Enter c (default 20): ") or 20)
60        T = float(input("Enter T (duration in seconds, default 10): ") or 10)
61        Fs = int(input("Enter Fs (sampling rate, default 44100): ") or 44100)
62        filename = input("Enter output filename (default output.wav): ") or "output.wav"
63
64        gwave(a, b, c, T, Fs, filename=filename)
65
66

```

Figure 2: Screenshot of python code

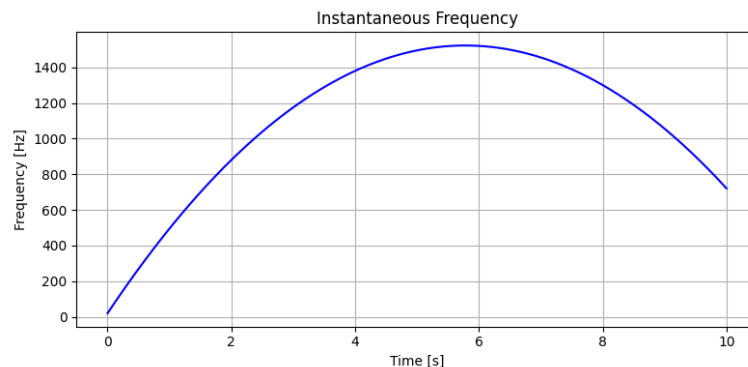


Figure 3: Instantaneous frequency v.s. time

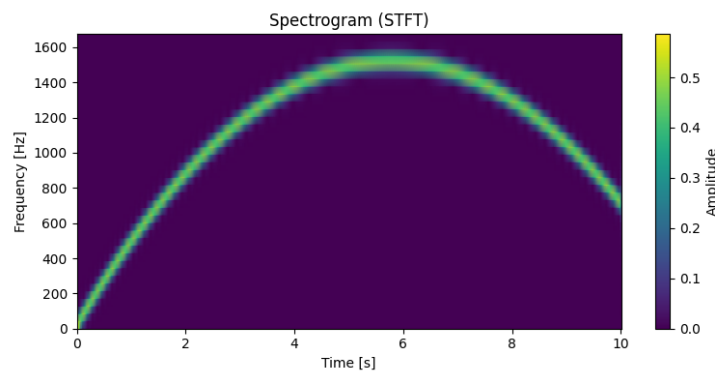


Figure 4: Spectrogram of STFT